

- JoshApp API Documentation
 - Overview
 - External APIs
 - DummyJSON Products API
 - Endpoints
 - Get Products
 - Internal Services
 - ApiService
 - Methods
 - fetchProducts(page: number = 1, limit: number = 10): Promise<ProductsResponse>
 - checkConnection(): Promise<boolean>
 - getMockProducts(page: number, limit: number): ProductsResponse
 - DatabaseService
 - Key Storage Items
 - Methods
 - registerUser(user: User): Promise<boolean>
 - loginUser(email: string, password: string): Promise<User | null>
 - getUserById(id: number): Promise<User | null>
 - updateUser(user: User): Promise<boolean>
 - deleteUser(id: number): Promise<boolean>
 - getAllMembers(): Promise<User[]>
 - Data Models
 - Product
 - ProductsResponse
 - User
 - Authentication Context
 - Context State
 - Methods
 - login(email: string, password: string): Promise<boolean>
 - register(user: Omit<User, 'id'>): Promise<boolean>
 - logout(): void
 - updateAccount(user: User): Promise<boolean>
 - deleteAccount(): Promise<boolean>
 - Implementation Notes

- [API Fallback Strategy](#)
- [AsyncStorage vs SQLite](#)
- [Mock Data Usage](#)
- [Debugging Tips](#)
 - [API Connection Issues](#)
 - [Authentication Problems](#)
 - [Data Persistence Issues](#)

JoshApp API Documentation

Overview

This document provides detailed technical information about the APIs and services used in the JoshApp React Native application. It covers both external API integrations and internal service implementations.

External APIs

DummyJSON Products API

The application fetches product data from the DummyJSON API.

Base URL: <https://dummyjson.com>

Endpoints

Get Products

```
GET /products
```

Query Parameters:

- **limit** (optional): Number of products to return (default: 30)
- **skip** (optional): Number of products to skip for pagination (default: 0)

Response Format:

```
{
  "products": [
    {
      "id": 1,
      "title": "Product Title",
      "description": "Product description",
      "category": "Category",
      "price": 19.99,
      "discountPercentage": 10.5,
      "rating": 4.5,
      "stock": 100,
      "tags": ["tag1", "tag2"],
      "brand": "Brand Name",
      "thumbnail": "thumbnail-url",
      "images": ["image-url-1", "image-url-2"]
    }
    // More products...
  ],
  "total": 100,
  "skip": 0,
  "limit": 30
}
```

Internal Services

ApiService

Located in `services/ApiService.ts`, this service handles external API communication.

Methods

`fetchProducts(page: number = 1, limit: number = 10): Promise<ProductsResponse>`

Fetches products from the DummyJSON API with pagination support.

Parameters:

- `page`: The page number to fetch (starts at 1)
- `limit`: Number of products per page

Returns: Promise resolving to a `ProductsResponse` object

Implementation Details:

- Calculates the `skip` parameter based on page and limit
- Makes a network request to the DummyJSON API
- Falls back to mock data if the request fails or if `useMockData` is true

`checkConnection(): Promise<boolean>`

Checks if the API is reachable.

Returns: Promise resolving to a boolean indicating connectivity status

`getMockProducts(page: number, limit: number): ProductsResponse`

Provides mock product data for offline use or testing.

Parameters:

- `page`: The page number to simulate
- `limit`: Number of products per page

Returns: A simulated ProductsResponse object from local mock data

DatabaseService

Located in `services/DatabaseService.ts`, this service manages local data storage using AsyncStorage.

Key Storage Items

- `@josh_app_users`: Array of user objects
- `@josh_app_current_user_id`: ID of the currently logged-in user

Methods

`registerUser(user: User): Promise<boolean>`

Registers a new user in the system.

Parameters:

- `user`: User object containing name, email, password, and optional fields

Returns: Promise resolving to a boolean indicating success

`loginUser(email: string, password: string): Promise<User | null>`

Authenticates a user by email and password.

Parameters:

- `email`: User's email address
- `password`: User's password

Returns: Promise resolving to the User object if authenticated, or null

`getUserById(id: number): Promise<User | null>`

Retrieves a user by their ID.

Parameters:

- `id`: User ID

Returns: Promise resolving to the User object if found, or null

`updateUser(user: User): Promise<boolean>`

Updates an existing user's information.

Parameters:

- `user`: Updated User object (must include ID)

Returns: Promise resolving to a boolean indicating success

`deleteUser(id: number): Promise<boolean>`

Deletes a user from the system.

Parameters:

- `id`: User ID to delete

Returns: Promise resolving to a boolean indicating success

`getAllMembers(): Promise<User[]>`

Retrieves all registered users.

Returns: Promise resolving to an array of User objects

Data Models

Product

```
interface Product {  
  id: number;  
  title: string;  
  description: string;  
  price: number;  
  discountPercentage: number;  
  rating: number;  
  stock: number;  
  brand: string;  
  category: string;  
  thumbnail: string;  
  images: string[];  
}
```

ProductsResponse

```
interface ProductsResponse {  
  products: Product[];  
  total: number;  
  skip: number;  
  limit: number;  
}
```

User

```
interface User {  
  id?: number;  
  name: string;  
  email: string;  
  password: string;  
  phone?: string;
```

```
location?: string;
}
```

Authentication Context

Located in `contexts/AuthContext.tsx`, this context provides authentication state and methods throughout the application.

Context State

```
interface AuthContextType {
  user: User | null;
  isLoading: boolean;
  login: (email: string, password: string) => Promise<boolean>;
  register: (user: Omit<User, 'id'>) => Promise<boolean>;
  logout: () => void;
  updateAccount: (user: User) => Promise<boolean>;
  deleteAccount: () => Promise<boolean>;
}
```

Methods

`login(email: string, password: string): Promise<boolean>`

Authenticates a user and updates the context state.

`register(user: Omit<User, 'id'>): Promise<boolean>`

Registers a new user and optionally logs them in.

`logout(): void`

Logs out the current user and clears the authentication state.

`updateAccount(user: User): Promise<boolean>`

Updates the current user's account information.

`deleteAccount(): Promise<boolean>`

Deletes the current user's account and logs them out.

Implementation Notes

API Fallback Strategy

The application implements a fallback strategy for API failures:

1. Attempt to fetch data from the live API
2. If the request fails or if network connectivity is unavailable, fall back to mock data
3. Mock data is structured identically to the API response to ensure consistent data handling

AsyncStorage vs SQLite

The application initially attempted to use SQLite for data persistence but encountered compatibility issues. AsyncStorage was chosen as the final solution for several reasons:

1. Better cross-platform compatibility
2. Simpler implementation with fewer dependencies
3. Adequate performance for the application's data volume
4. No type issues or complex setup required

The transition from SQLite to AsyncStorage required refactoring the DatabaseService while maintaining the same interface, ensuring that the rest of the application could continue to function without changes.

Mock Data Usage

Mock data is used in two scenarios:

1. When the `useMockData` flag is set to true in ApiService
2. As a fallback when network requests fail

The mock data structure exactly matches the DummyJSON API response format to ensure consistent data handling throughout the application.

Debugging Tips

API Connection Issues

To debug API connection issues:

1. Check the value of `useMockData` in `ApiService`
2. Look for console logs indicating API request failures
3. Verify network connectivity on the device

Authentication Problems

If authentication is not working correctly:

1. Check `AsyncStorage` for user data existence
2. Verify the email and password being used
3. Look for console logs from `DatabaseService` operations

Data Persistence Issues

If data is not persisting between app restarts:

1. Check that `AsyncStorage` operations are not failing silently
2. Verify that the correct storage keys are being used
3. Ensure that `AsyncStorage` is properly initialized

This API documentation complements the `README.md` file, providing in-depth technical details about the `JoshApp` implementation. Developers should refer to both documents for a complete understanding of the application.